

From Docker to Kubernetes: A History of Container Management

SHUYUAN ZHANG, The Chinese University of Hong Kong, Shenzhen, China

The paradigm of software deployment has undergone a profound metamorphosis over the last two decades, transitioning from monolithic physical servers to hypervisor-based virtual machines, and finally to operating system-level virtualization through containers. This paper presents an exhaustive historical and architectural analysis of container management, tracing its evolution from early Linux kernel primitives to the widespread adoption of Docker, and ultimately to the hegemony of Kubernetes as the de facto standard for cloud-native orchestration. By examining the architectural lineages of Google’s internal systems—Borg and Omega—this analysis elucidates the design philosophies that informed Kubernetes, contrasting centralized state management with decoupled choreography. Furthermore, it dissects the “orchestration wars” of 2014–2017, analyzing the technical, strategic, and governance factors that led to the obsolescence of competitors like Apache Mesos and Docker Swarm. An extensive taxonomy of contemporary Kubernetes scheduling algorithms is provided, detailing hardware-aware placement, multi-tenant network topologies, and distributed fog computing integration. Finally, this report explores the decoupling of container runtimes through the Container Runtime Interface (CRI) and Open Container Initiative (OCI), projecting the future trajectory of orchestration in the context of edge computing, artificial intelligence pipelines, and stateful enterprise workloads as of 2026.

CCS Concepts: • **Computer Systems Organization** → **Cloud Computing**; *Distributed Systems*; • **General and Reference** → Surveys and Overview.

Additional Key Words and Phrases: container orchestration, Kubernetes, Docker, cloud-native, scheduling, Borg, Omega

ACM Reference Format:

Shuyuan Zhang. 2026. From Docker to Kubernetes: A History of Container Management. *ACM Comput. Surv.* 1, 1, Article 1 (April 2026), 17 pages. <https://doi.org/1234567.1234567>

1 Introduction to Cloud-Native Computing and OS-Level Virtualization

The advent of cloud-native architectures has fundamentally decoupled application development from the underlying physical and virtual infrastructure. At the core of this systemic transformation is operating system-level virtualization, commonly referred to as containerization. Unlike traditional hypervisor-based virtualization (Virtual Machines), which abstracts hardware to run multiple full operating systems on a single physical host (see Figure 1), containerization abstracts the operating system itself, segmenting a single kernel into isolated user-space instances.

Containers leverage the native isolation capabilities of modern operating systems to sandbox processes. This approach incurs remarkably low overhead in resource consumption, as containers share the host machine’s kernel and do not require the instantiation, maintenance, or boot sequence of redundant guest operating systems. Consequently, containers offer near-native, “bare-metal”

Author’s Contact Information: Shuyuan Zhang, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, 225040535@link.cuhk.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7341/2026/4-ART1

<https://doi.org/1234567.1234567>

Hypervisor types

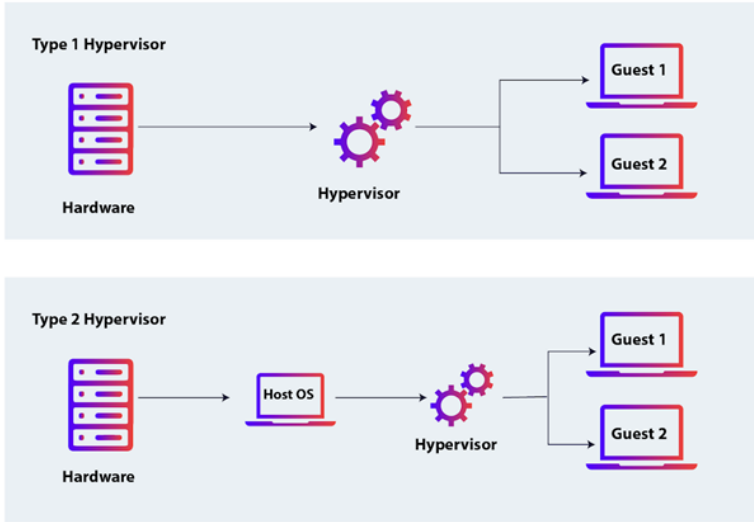


Fig. 1. Differences between Hypervisors and OS-Level Virtualization. Source: [6].

performance, rapid startup times measured in milliseconds, and unprecedented deployment density across cluster ecosystems.

The primary problem that containerization initially aimed to solve is the notorious “dependency hell” inherent in software engineering. Modern applications are highly composite, relying on specific versions of libraries, runtime environments, and adjacent microservices. A Python application, for example, might rely on a specific PostgreSQL data store and an Apache web server, each demanding distinct and potentially conflicting underlying library dependencies. By packaging an application alongside its exact dependencies into an immutable artifact, containers guarantee environmental parity across disparate development, testing, and production environments.

However, as container usage proliferated, it became evident that while the container runtime solved the problem of packaging and executing a single application on a single node, it did not address the complexities of managing hundreds of thousands of containers distributed across vast clusters of heterogeneous machines. The necessity to automate deployment, scaling, networking, and fault tolerance at scale catalyzed the development of container orchestration platforms. This report chronicles the trajectory from the democratization of the container image to the establishment of planetary-scale orchestration architectures.

2 The Genesis of Containerization and the Docker Revolution

The conceptual foundation of containerization predates the modern cloud era by several decades. The earliest precursor to process isolation can be traced to the chroot system call introduced in Unix Version 7 in 1979, which restricted a process’s file system access to a specific directory tree. This concept was substantially expanded by FreeBSD Jails in the early 2000s and Solaris Zones, which introduced robust partitioning of virtual environments, effectively pioneering OS-level compartmentalization.

In the Linux ecosystem, the foundational primitives for modern containers were established with the introduction of Control Groups (cgroups) in 2007—originally developed by engineers at Google—and the subsequent integration of Linux Namespaces in 2008. Namespaces provide fundamental isolation for various system resources, ensuring that a containerized process cannot perceive or interfere with processes outside its immediate sandbox. Specifically, the user namespace separates the container’s and host’s user databases, neutralizing root privilege escalation risks; the process namespace restricts visibility to only processes running within the container; and the network namespace provides the container with an autonomous network device and a virtual IP address.

Conversely, cgroups enforce resource accounting and limiting, preventing a single container from monopolizing shared physical resources such as CPU cycles, main memory, or disk I/O. These kernel features were amalgamated into LXC (Linux Containers), a user-space control package released in 2008 that represented the first standalone container runtime.

2.1 The Docker Architecture and File System Unification

Despite the availability of LXC, containerization remained a niche technology requiring deep systems expertise until the public debut of Docker in March 2013. Developed initially as an internal project at dotCloud, a Platform-as-a-Service startup founded by Kamel Founadi, Solomon Hybes, and Sebastien Pahl, Docker democratized OS-level virtualization by wrapping LXC in an accessible, developer-friendly command-line interface and a unified API. By version 0.9, Docker replaced LXC with its own Go-based component, `libcontainer`, fully controlling the interface to the Linux kernel.

Docker’s most profound breakthrough was not the invention of the container itself, but rather the standardization of the container image format and the streamlined workflow for building and distributing it. Docker introduced the `Dockerfile`, a declarative text script for assembling container images. The syntax utilized specific instructional keywords: `FROM` to designate a base operating system image, `MAINTAINER` for authorship tracking, `RUN` to execute shell commands and commit the resulting layer, `EXPOSE` to map network ports, `CMD` to define the default runtime process, and `ADD` or `COPY` to inject localized source code into the image payload.

Furthermore, Docker implemented the Advanced Multi-Layered Unification Filesystem (AuFS), a union mount copy-on-write filesystem. AuFS allowed Docker to construct images as a series of read-only layers. When a `RUN` instruction modified the system state, AuFS created a differential copy rather than duplicating the entire filesystem. Upon container instantiation, Docker overlaid a thin writable layer on top of the immutable image layers. This architecture drastically reduced storage requirements and accelerated deployment, as massive base layers (e.g., a core CentOS or Ubuntu image) could be shared across hundreds of diverse containers on the same host without duplication. Experimental support for other storage drivers, such as BTRFS, VFS, and Red Hat’s Device-Mapper, further expanded the flexibility of the platform. Table 1 summarizes the fundamental architectural differences between hypervisor-based Virtual Machines and OS-level Docker containers.

2.2 The Public Registry and Developer Ecosystem

Beyond the local runtime, Docker introduced the Docker Hub (or Central Index), a public registry that functioned analogously to Node’s NPM or Ruby’s RubyGems, but for complete application environments. Developers could instantly search, pull, and run pre-configured software stacks—from caching layers like Redis to web servers like Nginx—with guaranteed execution consistency.

The Docker CLI pipeline facilitated seamless interaction with the Docker daemon via REST-based web services. Through commands such as `docker ps` to observe running processes, `docker inspect` to extract virtual IP addresses, and `docker stop` to gracefully terminate execution, the

Table 1. Structural and architectural differences between traditional Virtual Machines and Docker Containers.

Architectural Feature	Hypervisor-Based VMs (Type 1 & 2)	OS-Level Virtualization (Docker Containers)
Abstraction Level	Hardware (Virtualizes the physical machine)	Operating System (Virtualizes the OS environment)
Guest OS Requirement	Required. Each VM boots a full, distinct operating system	Not required. Shares the host machine's OS kernel
Resource Overhead	Extremely high (Dedicated CPU, RAM, Storage per VM)	Minimal (Lightweight resource limits managed via cgroups)
Startup Time	Minutes (Requires full OS boot sequence)	Milliseconds to Seconds (Process instantiation)
Isolation Mechanism	Hardware virtualization extensions (Intel VT-x, AMD-V)	Linux kernel Namespaces (PID, NET, IPC, MNT, USER)
Storage Architecture	Monolithic virtual disks (VMDK, VHD, QCOW2)	Layered copy-on-write union filesystems (AuFS, OverlayFS)

barrier to entry for robust isolation was eradicated. By the release of version 1.0 in June 2014, Docker had catalyzed a tectonic shift toward microservice architectures, allowing developers to encapsulate single-responsibility components and deploy them rapidly. However, the Docker Engine was inherently designed to manage containers on a single host. As enterprises attempted to deploy complex topologies in production, the lack of native, multi-node orchestration became a critical bottleneck.

3 Ancestral Lineage: Google's Borg and Omega Systems

While the broader software industry was discovering containers via Docker in 2013, Google had already been orchestrating containerized workloads at an unprecedented planetary scale for a decade. To critically understand the architectural decisions underpinning modern orchestration algorithms, it is imperative to examine Google's internal proprietary systems: Borg and Omega.

3.1 Borg: The Monolithic Predecessor

Initiated around 2003–2004, Borg was Google's first unified container-management system. Prior to Borg, Google operated distinct systems for different workloads, namely Babysitter for long-running services and the Global Work Queue for batch processing. Borg was engineered to amalgamate these workloads onto shared physical infrastructure to solve a critical economic problem: massive hardware underutilization. By co-locating highly prioritized, latency-sensitive user-facing services (e.g., Google Search, Gmail) with low-priority, CPU-hungry batch jobs (e.g., MapReduce tasks), Borg reclaimed stranded resources, dynamically pausing batch processes during unexpected traffic spikes (as demonstrated in Figure 3).

Borg employed a highly centralized, monolithic architecture (Figure 2). The Borgmaster served as the single control plane, maintaining the cluster management logic, the state machines for all jobs and tasks, and the Paxos-based replicated storage system. The scheduler sequentially scanned a pending queue, assigning tasks to worker nodes based on multi-dimensional resource constraints. Furthermore, Borg relied on the Midas Package Manager (MPM) to build and deploy hermetic container images, striving to package all dependencies (statically linking binaries) to decouple the application from the host operating system entirely.

Despite its success, Borg suffered from significant usability constraints that heavily informed future orchestration designs. Tasks within Borg were grouped into “jobs” represented as rigid vectors of indices (e.g., task 0, task 1, task 2). This structural coupling made it exceedingly difficult to manage application state during rolling updates or machine failures. When a task in the middle of the vector terminated, it created “holes” in the array, disrupting applications that relied on range sharding based on the task index, often resulting in severe data unavailability. Furthermore, Borg assigned port numbers dynamically from the host machine to containers, forcing applications to rely on complex, non-standard naming services (the Borg Name Service) rather than traditional IP-based routing, requiring heavy modification of off-the-shelf software to handle arbitrary IP:port pairs.

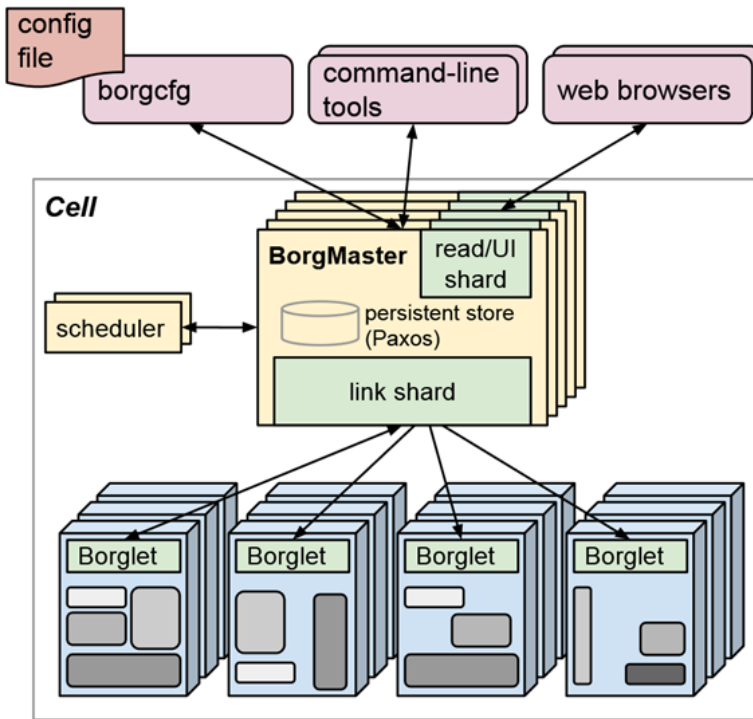


Fig. 2. High-level architecture of Borg. Taken from Figure 1 of Verma et al. [47].

3.2 Omega: Decentralization and Optimistic Concurrency

In 2013, Google introduced Omega, a second-generation cluster manager designed specifically to address the software engineering bottlenecks of the Borg ecosystem. As cluster sizes grew and specialized workloads emerged, Borg’s monolithic scheduler became a development chokepoint.

Omega introduced a fundamental architectural paradigm shift: it decentralized the scheduling logic (see Figure 4). Instead of funneling every placement decision and state change through a monolithic master, Omega utilized a centralized, Paxos-based transaction-oriented state store. Multiple independent, specialized schedulers (e.g., a batch scheduler, a service scheduler, a storage scheduler) operated as autonomous peers, directly reading from and writing to this shared state.

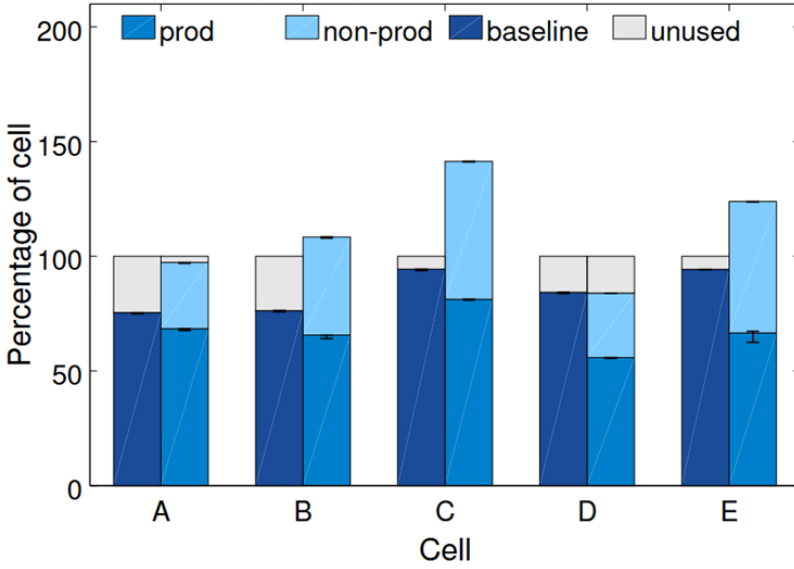


Fig. 3. Performance metrics of Borg. Taken from Figure 5a of Verma et al. [47].

To handle resource allocation conflicts among competing schedulers without implementing debilitating distributed locks, Omega employed Optimistic Concurrency Control (OCC). A scheduler would locally compute a placement decision based on its cached view of the cluster state and attempt an atomic commit to the central store. If another scheduler had claimed the same resources in the interim, the transaction failed, the first scheduler synchronized its state with the updated store, and the placement calculation was retried. This decoupled architecture proved highly scalable (with performance improvements highlighted in Figure 5), and the lessons learned regarding state synchronization, decentralized choreography, and API abstraction became the conceptual bedrock for Google's next iteration.

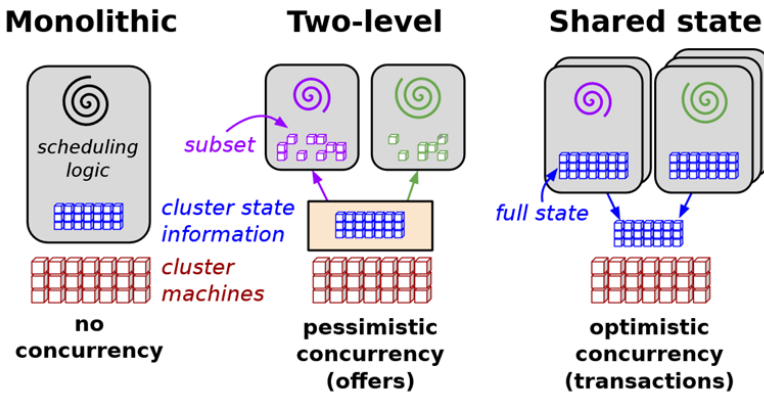


Fig. 4. High-level architecture of Omega. Taken from Figure 1 of Abd-El-Malek & Wilkes [3].

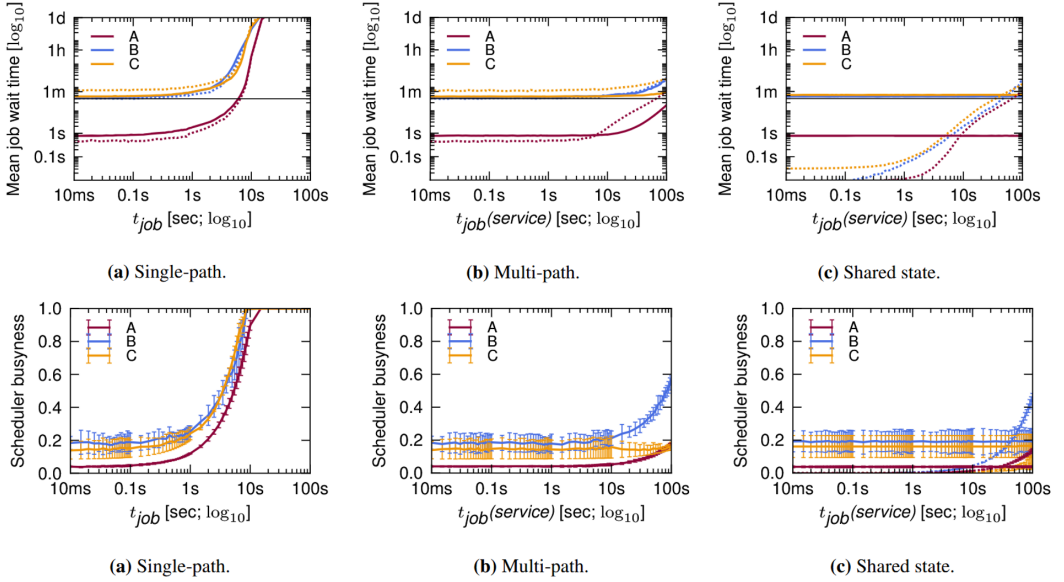


Fig. 5. Performance comparison of Omega. Taken from Figure 5 and 6 of Abd-El-Malek & Wilkes [3].

4 The Conception and Architectural Paradigm of Kubernetes

In the summer of 2013, Google engineers Craig McLuckie, Joe Beda, and Brendan Burns recognized that the external market was struggling with the exact container orchestration challenges Google had already solved internally. They proposed externalizing a new, open-source orchestrator built on the theoretical foundations of Borg and Omega, but designed explicitly to integrate with the rapidly standardizing Docker image format and cater to the emerging public cloud ecosystem.

The project was internally codenamed “Project Seven of Nine,” an homage to the *Star Trek: Voyager* character who is a friendly, reformed Borg drone. This legacy is permanently encoded in the Kubernetes logo, which features a seven-spoked ship’s wheel representing the original project name. Officially announced in June 2014 and releasing version 1.0 in July 2015, Kubernetes (derived from the Ancient Greek for “helmsman” or “pilot”) fundamentally revolutionized the orchestration landscape.

4.1 Epistemological Shifts: Pods, Labels, and IP-per-Pod

The designers of Kubernetes systematically rectified the architectural shortcomings discovered during a decade of operating Borg. Three primary innovations define the Kubernetes deployment paradigm:

- (1) **The Pod Abstraction:** Rather than scheduling individual, isolated containers, Kubernetes utilizes the “Pod” as the atomic unit of scheduling. A Pod is a resource envelope containing one or more tightly coupled containers that share a network namespace, IPC namespace, and storage volumes. This allows developers to compose composite applications using a primary application container alongside auxiliary “sidecar” containers—such as log forwarders, local caches, or service mesh proxies—without merging their binaries, preserving the single-responsibility principle.

- (2) **Labels and Selectors:** Moving completely away from Borg’s rigid job-task index vectors, Kubernetes introduced a dynamic, semantic querying system based on Labels. Labels are arbitrary key-value pairs (e.g., `role=frontend`, `tier=backend`, `stage=production`) attached to the metadata of objects. Controllers manage resources by continually querying the system for objects matching specific Label Selectors, allowing for highly flexible, overlapping topological groupings that can be modified dynamically at runtime without disrupting service architecture.
- (3) **IP-per-Pod Networking:** To resolve Borg’s crippling port-conflict issues and the necessity for proprietary name services, Kubernetes mandated an IP-per-Pod network model. Through Software-Defined Networking (SDN) overlays, every Pod is assigned a unique, routable IP address within the cluster. This aligns network identity directly with application identity, allowing off-the-shelf software to bind to standard ports (e.g., port 80 for HTTP traffic) natively, vastly simplifying service discovery, DNS routing, and load balancing.

4.2 Decoupled API Uniformity and Control by Choreography

Kubernetes synthesized the monolithic scheduling of Borg and the direct-state-access of Omega into a hybrid architecture centered on an API Server. The API Server acts as the sole interface to the persistent state store (etcd), hiding the underlying implementation while enforcing strict object validation, defaulting, and versioning. Every Kubernetes object is structured with absolute uniformity, requiring three fields: `ObjectMetadata` (containing UUIDs, versions, and labels), `Specification` or `Spec` (the user-defined desired state), and `Status` (the system-reported actual state).

Crucially, Kubernetes operates on a declarative model driven by independent reconciliation loops. Users do not issue imperative commands (e.g., “start container A on Node B”). Instead, they declare a desired state through controllers such as a `ReplicationController` (for run-forever services), a `DaemonSet` (for running an instance on every node), or a `Job` (for run-to-completion tasks).

Autonomous, decentralized controllers continuously observe the actual state of the cluster, compare it to the desired state, and execute actions to reconcile the delta (a high-level overview of this architecture is depicted in Figure 6). This philosophy, termed “control through choreography,” creates an emergent behavior that makes the system exceptionally resilient to component failures and network partitions. Decoupling components further ensures that systems like the horizontal pod autoscaler can function purely by adjusting the numerical value in a `ReplicaSet`’s `Spec`, without needing to understand the underlying logic of how containers are actually provisioned by the kubelet.

5 The Container Orchestration Wars (2014–2017)

The release of Kubernetes precipitated a fierce industry conflict to establish the dominant standard for container orchestration. Between 2014 and 2017, the market fractured into competing factions, primarily contested by Docker Swarm, Apache Mesos, and Kubernetes.

5.1 Docker Swarm: Simplicity vs. Scalability

Introduced natively by Docker Inc. in late 2015, Swarm was marketed as the simplistic, frictionless clustering solution for existing Docker environments. Swarm’s primary architectural advantage was its strict adherence to the standard Docker API; any developer tool or Continuous Integration pipeline that communicated with a single Docker daemon could instantly communicate with a multi-node Swarm cluster.

Swarm utilized a centralized cluster manager module that implemented a filtering and scheduling strategy phase. It supported three primary scheduling strategies: the spread strategy (distributing

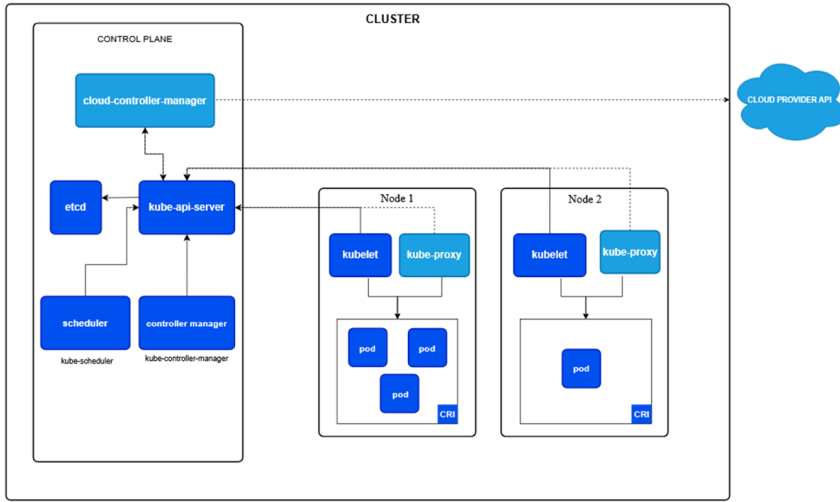


Fig. 6. Kubernetes Architecture. Source: [36].

containers across the least-loaded nodes to maximize redundancy), the binpack strategy (packing containers onto the most-loaded node to preserve contiguous free infrastructure for large workloads), and a standard random strategy. Docker Inc. aggressively promoted Swarm’s raw performance, commissioning benchmark tests that claimed Swarm could instantiate containers up to 98 times faster than Kubernetes under 100% cluster load.

However, Swarm suffered from critical architectural limitations. It lacked advanced deployment features—such as robust native autoscaling, automated self-healing, advanced Service Function Chaining, and granular Role-Based Access Control (RBAC). Furthermore, its reliance on the Raft consensus algorithm for maintaining cluster state across manager nodes severely limited its theoretical scalability compared to enterprise alternatives.

5.2 Apache Mesos: Hyperscale Data Center Management

Conversely, Apache Mesos approached the orchestration problem from the perspective of global data center resource allocation rather than application-centric container deployment. Originally developed as a research project at UC Berkeley in 2009, Mesos functioned as a distributed systems kernel, abstracting and aggregating CPU, memory, and storage across tens of thousands of nodes.

Mesos utilized a complex two-level scheduling architecture. The central Mesos master utilized an algorithm known as Dominant Resource Fairness (DRF) to equitably offer available cluster resources to application-level frameworks. Frameworks such as Hadoop for big data, Spark for data analytics, or Marathon and Aurora specifically built for container orchestration, would then accept these resource offers and execute their specific task scheduling logic independently.

While Mesos demonstrated unparalleled scalability—famously utilized by Twitter (launching Apache Aurora) and Apple (powering the Siri backend) to manage massive infrastructure footprints—it was notoriously complex to install, configure, and maintain. It was fundamentally designed for hyperscale infrastructure operators running mixed batch and service workloads, not for the every-day application developer seeking rapid, declarative microservice deployment. The comparative architecture and mechanisms of these primary orchestration engines are outlined in Table 2.

Table 2. Comparative Architecture and Mechanisms of the Primary Container Orchestration Platforms (2014–2017).

Orchestration Engine	Architectural Paradigm	Primary Scheduling Logic	Core Abstraction
Docker Swarm	Centralized, monolithic manager	Spread, Binpack, Random	Container
Apache Mesos	Two-level, decentralized frameworks	Dominant Resource Fairness (DRF)	Frameworks / Tasks
Kubernetes	API-centric decoupled choreography	Predicates & Priorities (Filtering/Ranking)	Pod

5.3 The Strategic Triumph of Kubernetes and the CNCF

By the end of 2017, the orchestration wars had effectively concluded, with Kubernetes emerging as the undisputed victor and de facto industry standard. The surrender was formalized when major cloud providers (AWS announcing EKS, Microsoft announcing AKS), and even rival orchestration vendors like Docker (incorporating Kubernetes into Docker Enterprise) and Mesosphere (renaming to D2iQ and sunseting DC/OS), announced native support for Kubernetes. This hegemony was achieved through a confluence of superior engineering, strategic alliances, and open-source governance.

The most decisive strategic maneuver occurred simultaneously with the release of Kubernetes 1.0 in July 2015. Recognizing that enterprises would hesitate to adopt a critical infrastructure layer controlled solely by a single vendor with competing commercial interests (a criticism frequently leveled at Docker Inc.), Google donated the Kubernetes codebase, intellectual property, and trademark to the newly formed Cloud Native Computing Foundation (CNCF).

The CNCF, operating as a subsidiary of the Linux Foundation, was established with cross-industry backing from tech titans including IBM, Red Hat, Microsoft, VMware, Cisco, and Intel. This coalition effectively neutralized the threat of vendor lock-in and provided a neutral ground for fierce competitors to collaborate on shared infrastructure. In contrast to Docker Inc., which struggled to build a collaborative community due to its tight corporate control and aggressive monetization strategies, Kubernetes embraced a completely open, transparent, and vendor-neutral governance model.

The velocity of the Kubernetes community rapidly outpaced its rivals. By March 2018, when Kubernetes became the first project to formally “graduate” from the CNCF, the statistics were staggering. It was the second most active open-source project in the world behind the Linux kernel. Since joining the CNCF, the contributor base exploded by over 996%, accumulating over 74,680 contributors, 314,000 code commits, and 3.4 million overall contributions. Furthermore, the initial monopoly held by Google and Red Hat on codebase commits dropped from 83% to 46%, indicating healthy, widespread corporate adoption. This sheer volume of engineering effort, characterized by what industry analysts termed “enlightened self-interest,” created an inescapable gravitational pull that starved competing platforms of developer attention and cemented Kubernetes’ dominance.

6 A Comprehensive Taxonomy of Kubernetes Resource Scheduling

To fully comprehend the dominance of Kubernetes, an academic examination of its internal resource management and scheduling mechanisms is required. The Kubernetes orchestrator undertakes a complex, multi-dimensional NP-hard optimization problem: assigning heterogeneous microservices

to dynamic hardware resources while adhering to strict Quality of Service (QoS) guarantees, minimizing interference, and maximizing infrastructure utilization.

The assignment of pods to physical or virtual worker nodes is executed by the kube-scheduler (the Default Scheduler). The scheduler operates as an independent controller that watches the etcd datastore via the API Server for unbound pods, placing them into a podQueue. The scheduling logic follows a strictly defined, two-stage sequential process for each pod extracted from the queue:

- (1) **Filtering (Predicates):** In this stage, the scheduler evaluates all available nodes in the cluster against the specific requirements of the pod. Predicates are Boolean functions that eliminate unsuitable nodes. Common pre-defined policies include PodFitsResources (ensuring the node has sufficient unallocated compressible resources like CPU and non-compressible resources like RAM), PodFitsHostPorts (checking for network port collisions on the host interface), and CheckNodeCondition (verifying the underlying node's health metrics and network availability). Node selection can also be forced via hard node affinity rules driven by labels (e.g., disktype:ssd).
- (2) **Ranking (Priorities):** The candidate nodes that survive the filtering stage are subjected to complex scoring algorithms. The scheduler computes a weighted score (from 0 to 10) for each viable node to optimize the final placement. The LeastRequestedPriority policy favors nodes with the highest percentage of free resources to distribute load, whereas the BalanceResourceAllocation policy attempts to keep CPU and memory usage strictly proportional across the cluster. The SelectorSpreadPriority ensures that replicas of the same service are dispersed across multiple physical failure domains to guarantee high availability. The pod is subsequently bound to the node with the highest aggregate score.

As Kubernetes has penetrated diverse enterprise environments—from highly constrained edge devices to massive GPU clusters—academic and industrial research has massively expanded the scope of scheduling methodologies. A taxonomy of this research reveals five distinct domains of ongoing optimization: Infrastructure, Cluster, Scheduling Algorithms, Application, and Performance.

6.1 The Infrastructure Domain: Hardware and Virtual Layer Optimization

The default Kubernetes scheduler is generally agnostic to the specific nuances of underlying hardware, treating nodes as generic pools of CPU and RAM. However, advanced workloads necessitate hardware-aware scheduling.

At the Physical Layer, researchers have extended Kubernetes to manage heterogeneous architectures, such as clusters blending powerful x86 servers with low-power ARM-based Single-Board Computers (SBCs) like Raspberry Pis. Systems like Tarema implement task and infrastructure profiling to dynamically allocate scientific workflows based on the specific hardware properties of nodes, reducing runtimes by over 4.5% compared to default shortest-job-next schedulers. Furthermore, the proliferation of hardware accelerators requires sophisticated management. Schedulers such as Gaia and Kube-Knot have been engineered to divide physical GPUs into virtual partitions, safely co-locating pods on Nvidia hardware while monitoring real-time utilization correlations to prevent out-of-memory crashes during deep learning training. Frameworks like GPAS even integrate sidecar monitors to adaptively reallocate GPUs during the training process based on live telemetry.

At the Virtual Layer, research focuses predominantly on mitigating “noisy neighbor” container interference and optimizing software-defined network topologies. While cgroups provide isolation, they do not perfectly shield applications from contention over shared L3 processor caches or memory bandwidth. Schedulers like KubeSphere implement multi-tenant, policy-driven DRF logic to ensure fairness, while Skynet builds runtime models mapping performance-level objectives to

hardware metrics (IPC, reads, writes) to decrease interference-driven objective violations by up to 7.4×. Regarding networking, default Kubernetes assumes a flat network topology, which is highly inefficient for hierarchical fog/edge environments. Network-aware schedulers, such as ElasticFog and Polaris, construct dynamic cluster topology graphs, collecting real-time network traffic and Round Trip Time (RTT) data to provision pods near data sources, minimizing latency for IoT and Service Function Chaining (SFC) workloads.

6.2 The Cluster Domain: Multi-Cluster Federation and Edge Scheduling

The centralized nature of the default kube-scheduler creates scalability bottlenecks and single points of failure in massive deployments. Consequently, architectural research in the Cluster Domain explores alternative paradigms. Multi-level and distributed scheduling architectures have gained prominence. Rather than a monolithic dispatcher, frameworks like the Multi-Agent System (MAS) delegate the filtering and ranking operations directly to agents running on the processing nodes, reducing centralized scheduling delays by a factor of nine.

A 2022 comprehensive survey by Carmen Carrión [1] emphasized the architectural bottleneck of Kubernetes' centralized control plane and the necessity for collaborative, multi-cluster coordination. The necessity to avoid vendor lock-in and operate across distinct geographic regions spurred the development of Multi-Cluster and Federation schedulers like KubeFed and mck8s, which orchestrated workloads across entirely separate control planes. More recently, frameworks such as Karmada have been widely adopted to provide centralized multi-cloud management and advanced cross-cluster traffic scheduling without requiring modifications to native applications.

Furthermore, the Environment subdomain highlights the adaptation of Kubernetes into Fog and Edge computing architectures. Edge computing places storage and compute resources immediately adjacent to IoT data generation sources. Standard Kubernetes is often too resource-heavy for these environments. CNCF-incubated distributions like K3s (which replaces the heavy etcd datastore with a lightweight SQLite backend) and KubeEdge allow developers to schedule workloads across the cloud-to-edge continuum using identical APIs. For edge deployments where latency and network constraints are critical, specialized schedulers like Polaris have emerged. Polaris introduces Cluster Topology Graphs and Service Graphs to make scheduling decisions that are explicitly aware of network Quality of Service (QoS) and Service Level Objectives (SLOs) at the edge.

6.3 The Scheduling and Application Domains: Algorithms, FaaS, and Gang Scheduling

The computational logic utilized by custom schedulers spans a wide mathematical spectrum. While default Kubernetes relies on low-complexity Heuristics (First-Fit, Round-Robin), modern research implements advanced models. Mathematical Modeling techniques like Integer Linear Programming (ILP) provide optimal placements but suffer from high computational overhead. Meta-heuristics, such as Genetic Algorithms (GA) and Particle Swarm Optimization (PSO), balance placement efficiency with computational speed. More recently, Machine Learning algorithms like the DL2 deep learning-driven scheduler leverage LSTM neural networks to predict future resource utilization spikes, pre-emptively rescheduling pods before node exhaustion occurs.

Perhaps the most significant evolution stems from the call for advanced, context-aware scheduling algorithms that account for energy consumption, carbon footprint, and dynamic workload prediction [1]. By 2025, environmental sustainability became a primary scheduling objective. Modern experimental schedulers execute both temporal and spatial workload shifting, migrating non-critical computing tasks to times or data centers where low-carbon electricity is most abundant. Frameworks now utilize machine learning alongside models like NSGA-II to dynamically balance power consumption. For instance, the PAX scheduler utilizes Bayesian optimization to enable

Table 3. Comprehensive Taxonomy of Kubernetes Scheduling Research Domains and Frameworks.

Scheduling Domain	Optimization Focus & Metrics	Representative Frameworks	Academic
Physical Layer	Hardware awareness, GPUs, CPU architecture, power consumption	Tarema, Polaris, Gaia, Kube-Knot, GPAS	
Virtual Layer	Multi-tenancy isolation, network latency (RTT), Service Function Chaining	KubeSphere, Skynet, ElasticFog	
Cluster Architecture	Decentralized multi-agent dispatch, multi-cluster federation	MAS, KubeFed, Foggy, mck8s, Karmada	
Algorithms	Heuristics, ILP, Meta-heuristics (GA, ACO), Machine Learning (LSTM)	DL2, OASIS, PAX	
Application Logic	Gang scheduling (all-or-nothing), Serverless (FaaS) pre-warming	DRAGON, Knative, Pigeon	

energy-aware pod placement, minimizing operational carbon emissions by dynamically throttling workloads, while also mitigating embodied carbon emissions associated with hardware [45].

In the Application Domain, scheduling strategies must account for specific architectural demands. Standard Kubernetes evaluates pods sequentially, which is highly detrimental for tightly coupled distributed applications like Machine Learning training rings where 100 pods might be required simultaneously. To rectify this, earlier researchers implemented Gang Scheduling controllers (e.g., DRAGON, OASIS) that treat dependent pods as a single schedulable unit. Building upon these gaps identified in 2022 [1], Kubernetes v1.35 officially introduced the native Workload API (`scheduling.k8s.io/v1alpha1`) in late 2025, delivering built-in gang scheduling capabilities [46] to prevent resource deadlocks. Concurrently, tools like Kueue have matured into robust cloud-native job queueing systems for batch workloads.

Similarly, the rise of Function-as-a-Service (FaaS) serverless paradigms (supported by frameworks like Knative) demands extreme latency reduction. Custom schedulers like Pigeon bypass standard Kubernetes limitations by maintaining static pools of pre-warmed containers, drastically reducing cold-start latency. Table 3 provides a comprehensive taxonomy of these scheduling research domains and representative frameworks.

7 The Evolution of Container Runtimes and the Open Container Initiative

As Kubernetes matured into the undisputed industry standard, its architectural relationship with Docker underwent a critical and highly scrutinized transformation. Initially, the Kubernetes kubelet relied entirely on the Docker Engine API to handle container instantiation, lifecycle management, and image pulling. However, the Docker Engine was a heavy, monolithic daemon designed primarily for direct human interaction. It encompassed networking management, storage volumes, and image building utilities—features that were either entirely redundant or actively conflicting when operating within a managed Kubernetes cluster.

To achieve runtime agnosticism, improve system performance, and reduce the attack surface of the worker nodes, the Kubernetes community established the Container Runtime Interface (CRI) in 2016. The CRI defined a standard gRPC API that allowed the Kubernetes kubelet to communicate

seamlessly with any compliant container runtime without requiring direct recompilation of the core Kubernetes codebase.

Concurrently, the industry pushed to standardize the core container image format and execution environment through the Open Container Initiative (OCI), preventing market fragmentation. This enabled the development of lightweight, highly optimized runtimes focused exclusively on container execution, bypassing the bulk of the Docker Engine.

Docker Inc. responded by extracting its core container execution logic into a standalone, OCI-compliant daemon called `containerd`, which was subsequently donated to the CNCF. Simultaneously, Red Hat led the development of CRI-O, an ultra-lightweight runtime engineered explicitly to satisfy the Kubernetes CRI with absolute minimal overhead. By standardizing primarily on `containerd` and CRI-O, Kubernetes effectively outgrew its historical dependency on the Docker Engine. This decoupling culminated in the formal deprecation and eventual removal of `docker shim` (the legacy translation layer that allowed Kubernetes to speak directly to the Docker daemon) from the Kubernetes codebase in 2022. This event signaled the final phase of Docker's transition from a foundational production runtime layer to primarily a localized developer toolset and image construction utility.

8 Contemporary Trends and Future Trajectories (2020s–2026)

Moving into the mid-2020s, the operational scope of Kubernetes has expanded significantly beyond its original mandate of managing stateless, twelve-factor web microservices. The 2025 and 2026 iterations of the CNCF Annual Cloud Native Survey indicate that Kubernetes is now the omnipresent substrate for enterprise IT, utilized in production by over 82% of all container users. Several macro-trends define the current state and future trajectory of the platform:

8.1 Convergence with Artificial Intelligence and Agentic Workloads

The explosion of Generative AI, Large Language Models (LLMs), and autonomous AI agents has fundamentally altered infrastructure demands. Modern AI pipelines require the seamless, highly coordinated execution of immense distributed data processing (e.g., Apache Spark ETL jobs), highly parallelized GPU-accelerated training jobs, and highly elastic, low-latency inference services.

Rather than deploying disparate, siloed infrastructure for big data, machine learning, and application serving, enterprises are rapidly converging these varied workloads onto a unified Kubernetes foundation. The platform's immense extensibility allows operators to orchestrate specialized hardware accelerators (GPUs, TPUs) with granular precision, while advanced Kubernetes scheduling frameworks manage the burst scaling required by massive AI inference runs. By 2026, 66% of organizations hosting generative AI workloads utilize Kubernetes as their primary infrastructure orchestration layer, shifting the paradigm from request/response APIs to long-running, stateful reasoning loops.

8.2 Stateful Workloads, Data Sovereignty, and Day Two Reliability

Historically, databases, message queues, and storage-heavy applications were strictly maintained on legacy virtual or physical infrastructure due to the ephemeral, transient nature of containers. However, the introduction of the Container Storage Interface (CSI) and highly robust persistent volume paradigms has catalyzed the mass migration of stateful workloads directly into Kubernetes clusters. Recent telemetry reveals a near-parity between stateless (63%) and stateful (58%) application deployments, signifying absolute confidence in Kubernetes as the backbone for business-critical systems.

Furthermore, shifting geopolitical realities and complex regulatory frameworks have elevated data sovereignty to a strategic enterprise imperative. Kubernetes enables consistent, declarative

multi-cloud and hybrid-cloud deployments. This allows organizations to seamlessly migrate highly sensitive stateful applications to compliant, geographically bound sovereign data centers while maintaining identical operational workflows, GitOps pipelines, and internal developer platforms. Consequently, operational maturity in 2026 is measured by “Day Two” reliability: cross-cluster consistency, volume failure recovery, and the ability to restore stateful services seamlessly across regions.

8.3 FinOps and Continuous Optimization

As Kubernetes footprints scale exponentially across global enterprises, the complexity of cost management (Kubernetes FinOps) and active resource optimization has intensified drastically. While the platform excels at automated deployment and horizontal pod autoscaling, human developers frequently over-provision CPU and memory requests to prevent out-of-memory terminations, leading to substantial resource wastage and inflated cloud expenditure. The future trajectory of Kubernetes involves bridging the gap between rapid deployment speed and post-deployment operational efficiency. This requires the deep integration of AI-driven optimization control loops that continuously analyze real-world telemetry data and autonomously tune resource constraints, optimizing pod placements in real-time to match fluctuating demand without manual intervention.

9 Conclusion

The evolution from Docker to Kubernetes represents one of the most rapid, pervasive, and consequential architectural shifts in the history of computer science. Operating system-level virtualization, democratized by Docker, decisively solved the localized problem of application packaging, environmental consistency, and dependency management through its layered filesystem and standardized image format. However, it was Kubernetes—born from the crucible of Google’s massive-scale Borg and Omega systems, and propelled by the vendor-neutral governance of the Cloud Native Computing Foundation—that definitively solved the distributed systems orchestration problem.

By abandoning rigid imperative configurations in favor of declarative, label-driven reconciliation loops and a highly decoupled API, Kubernetes established an infinitely extensible, fault-tolerant control plane. It decisively won the container orchestration wars not merely through raw throughput or benchmark performance, but by providing an open, standardized, community-driven framework that the entire technology industry could collaborate upon.

As of 2026, Kubernetes has transcended its origins as a mere container scheduler. Supported by advanced algorithmic research in scheduling topologies and hardware-aware placement, it functions as the universal abstraction layer for planetary-scale computing. Orchestrating everything from ephemeral serverless functions to sprawling, GPU-intensive generative AI pipelines, and bridging the gap from sovereign cloud data centers to constrained edge devices, the choreographed architecture of Kubernetes guarantees its enduring position as the foundational operating system of the modern cloud era.

References

- [1] A Comprehensive Feature Comparison Study of Open-Source Container Orchestration Frameworks. <https://arxiv.org/abs/2002.02806>
- [2] A history of low-level Linux container runtimes. <https://opensource.com/article/18/1/history-low-level-container-runtimes>
- [3] Abd-El-Malek, M., & Wilkes, J. Omega: flexible, scalable schedulers for large compute clusters. <https://cs.brown.edu/people/malte/pub/papers/2013-eurosys-omega.pdf>
- [4] Isam Al Jawarneh. Container Orchestration Engines: A Thorough Functional and Performance Comparison. <https://isamaljawarneh.github.io/pubs/ICC19.pdf>

- [5] Application Container Placement and Management across Multi-cloud Clusters. https://www.researchgate.net/publication/326985958_Application_Container_Placement_and_Management_across_Multi-cloud_Clusters
- [6] AppViewX. Hypervisor. <https://www.appviewx.com/education-center/hypervisor/>
- [7] Aqua Security. The History of Kubernetes. <https://www.aquasec.com/blog/kubernetes-history-how-it-conquered-cloud-native-orchestration/>
- [8] Aqua Security. A Brief History of Containers: From the 1970s Till Now. <https://www.aquasec.com/blog/a-brief-history-of-containers-from-1970s-chroot-to-docker-2016/>
- [9] Borg, Omega, and Kubernetes - Google Research. <https://research.google.com/pubs/archive/44843.pdf>
- [10] Borg, Omega, and Kubernetes - ACM Queue. <https://queue.acm.org/detail.cfm?id=2898444>
- [11] Borg, Omega, and Kubernetes - ResearchGate. https://www.researchgate.net/publication/297595470_Borg_Omega_and_Kubernetes
- [12] Cloud Native Computing Foundation announces new container technologies alignment. <https://www.cncf.io/announcements/2015/06/21/new-cloud-native-computing-foundation-to-drive-alignment-among-container-technologies/>
- [13] CNCF. Demystifying containers - part II: container runtimes. <https://www.cncf.io/blog/2019/07/15/demystifying-containers-part-ii-container-runtimes/>
- [14] CNCF. Kubernetes Project Journey Report. <https://www.cncf.io/reports/kubernetes-project-journey-report/>
- [15] CNCF announces Kubernetes as first graduated project. <https://www.cncf.io/announcements/2018/03/06/cloud-native-computing-foundation-announces-kubernetes-first-graduated-project/>
- [16] CNCF. Kubernetes Fuels AI Growth; Organizational Culture Remains the Decisive Factor. <https://www.cncf.io/blog/2026/01/20/kubernetes-fuels-ai-growth-organizational-culture-remains-the-decisive-factor/>
- [17] CNCF Annual Cloud Native Survey. https://www.cncf.io/wp-content/uploads/2026/01/CNCF_Annual_Survey_Report_final.pdf
- [18] Comparison: Docker Swarm vs. Kubernetes vs. Apache Mesos vs. CoreOS Fleet. <https://medium.com/@haroldfinch01/comparison-docker-swarm-vs-kubernetes-vs-apache-mesos-vs-coreos-fleet-280d92f2f0b1>
- [19] Container Wars: Benchmark Test Pits Docker's Swarm Platform Against Kubernetes. <https://www.channelfutures.com/cloud/container-wars-benchmark-test-pits-docker-s-swarm-platform-against-kubernetes>
- [20] Docker (software) - Wikipedia. [https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))
- [21] Docker Swarm vs Kubernetes: Key Differences, Pros & Cons. <https://talent500.com/blog/docker-swarm-vs-kubernetes-comparison/>
- [22] Docker Swarm vs Kubernetes: How to choose a container orchestration tool. <https://circleci.com/blog/docker-swarm-vs-kubernetes/>
- [23] Exploiting content similarity to address cold start in container deployments. <https://dna-pubs.cs.columbia.edu/uploads/b52ddfb-d-063b-46ec-8466-ac0d4accb3ff.pdf>
- [24] From Docker to Kubernetes: Building Resilient CI/CD for Node.js and Next.js Applications. https://www.researchgate.net/publication/394235135_From_Docker_to_Kubernetes_Building_Resilient_CICD_for_Nodejs_and_Nextjs_Applications
- [25] Getting Started with Kubernetes | A Brief History of Cloud-native Technologies. https://www.alibabacloud.com/blog/getting-started-with-kubernetes-%7C-a-brief-history-of-cloud-native-technologies_595894
- [26] How Kubernetes came to be: A co-founder shares the story. <https://cloud.google.com/blog/products/containers-kubernetes/from-google-to-the-world-the-kubernetes-origin-story>
- [27] How did they ever come up with that kooky 'Kubernetes' name? <https://www.geekwire.com/2016/ever-come-kooky-kubernetes-name-heptio/>
- [28] How Kubernetes Won the Container Orchestration War. <https://www.izumanetworks.com/blog/how-k8s-won-the-orchestration-war/>
- [29] IBM. The History of Kubernetes. <https://www.ibm.com/think/topics/kubernetes-history>
- [30] Container Management: Kubernetes vs Docker Swarm, Mesos + Marathon, Amazon ECS - Platform9. <https://platform9.com/media/kubernetes-comparison-ebook.pdf>
- [31] Kubernetes: How a Rejected Internal Project Became a Global Standard. <https://www.opensourcerers.org/2023/03/20/kubernetes-how-a-rejected-internal-project-became-a-global-standard/>
- [32] Kubernetes - Wikipedia. <https://en.wikipedia.org/wiki/Kubernetes>
- [33] Borg: The Predecessor to Kubernetes. <https://kubernetes.io/blog/2015/04/borg-predecessor-to-kubernetes/>
- [34] Kubernetes vs Docker Swarm vs Apache Mesos - Codefresh. <https://codefresh.io/blog/kubernetes-vs-docker-swarm-vs-apache-mesos/>
- [35] Kubernetes (K8s): From Orchestration War To Platform Showdown To OS Standard? <https://www.forrester.com/blogs/kubernetes-k8s-from-orchestration-war-to-platform-showdown-to-os-standard/>
- [36] Kubernetes Documentation. Kubernetes Architecture. <https://kubernetes.io/docs/concepts/architecture/>

- [37] Large-scale cluster management at Google with Borg. <https://research.google.com/pubs/archive/43438.pdf>
- [38] Omega: flexible, scalable schedulers for large compute clusters. <https://www.pdl.cmu.edu/SDI/2013/slides/wilkes-sdi.pdf>
- [39] Omega: flexible, scalable schedulers for large compute clusters - Brown CS. <https://cs.brown.edu/people/malte/pub/papers/2013-eurosys-omega.pdf>
- [40] Overview of Different Container Runtimes - CTO.ai. <https://cto.ai/blog/overview-of-different-container-runtimes/>
- [41] Performance Evaluation of Container Orchestration Tools in Edge Computing Environments. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10143384/>
- [42] The great migration: Why every AI platform is converging on Kubernetes. <https://www.cncf.io/blog/2026/03/05/the-great-migration-why-every-ai-platform-is-converging-on-kubernetes/>
- [43] The State of Cloud Native Optimization 2026. <https://akamas.io/resources/the-state-of-cloud-native-optimization-2026/>
- [44] Top 5 Kubernetes Trends to Watch at KubeCon Amsterdam 2026. <https://www.nutanix.com/blog/kubernetes-trends-kubecon-amsterdam-2026>
- [45] Towards Performance and Energy Aware Kubernetes ... - HotCarbon. <https://hotcarbon.org/assets/2025/paper-80.pdf>
- [46] Kubernetes v1.35: Introducing Workload Aware Scheduling ... <https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/>
- [47] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15). Association for Computing Machinery, New York, NY, USA, Article 18, 1–17. <https://doi.org/10.1145/2741948.2741964>
- [48] What 500+ Infrastructure Leaders Say About the Future of Kubernetes. <https://portworx.com/blog/kubernetes-enterprise-adoption-trends/>
- [49] Why Kubernetes is winning the container war. <https://www.infoworld.com/article/2248699/why-kubernetes-is-winning-the-container-war.html>

Received 2026